

- **Version Control:** Because this is a large project, consider storing your VBA in an external `` .bas `` and `` .frm `` files and track changes with version control software if possible.

With these modules, userforms, and best practices in place, you'll have a **solid foundation** for your PM Scheduling System in VBA. Continue to refine the UI/UX, handle edge cases, and optimize as your dataset and requirements grow.



I want you to give me the entirety of phase 1 and 2- I want all VBA code as well as extremely specific step by step instructions on how to set up everything from the ActiveX controls to the userform.

Mapping the process

I'm organizing the code for "modGlobal," "modUtilities," "modDataUpload," "frmUpload," and forming step-by-step instructions to set up sheets, userform, controls, and macros.

Laying out the groundwork

I'm detailing the setup for the "Dashboard" sheet with an upload button and data storage sheets "Account Data," "Call Logs," "Reminders," and "Change Log" for phases 1 and 2.

Integrating the code

I'm considering merging the "Dashboard_UploadAccounts" macro into "modDashboard" for simplicity, ensuring it aligns with data upload requirements and streamlines the setup process.

Below is a **comprehensive guide** for implementing **Phase 1** and **Phase 2** of your project, including **all relevant VBA code** and extremely **detailed, step-by-step setup instructions**. These phases focus on:

1. **Workbook Setup & Core Modules** (Phase 1)
2. **Upload & Preview of Accounts** (Phase 2)

Because you asked for "all VBA code as well as extremely specific step by step instructions on how to set up everything from the ActiveX controls to the userform," the sections below are structured as follows:

1. **Worksheet Setup** (Sheets & required columns)
2. **Creating the VBA Modules** (and the code that goes in each)
3. **Creating and Configuring the Dashboard** (with an ActiveX button)
4. **Creating the "Upload Accounts" UserForm** (`` frmUpload ``) and its controls/code

By the end, you will have a working file where the user can press an **"Upload Accounts"** button, browse for an Excel file, **preview** new vs. duplicate accounts, and **import** them into your main workbook.

1. WORKSHEET SETUP

To support Phases 1 and 2, you need the following sheets in your workbook:

1. Dashboard

- This will serve as your main user interface.
- We'll put an **ActiveX command button** here for "Upload Accounts."

2. Account Data

- Stores the unique combination of Relationship#, Master Account ID, and Account#.
- **Minimum required columns** (left to right). You can add more, but for Phase 2 we only need these columns:
 1. `Relationship#`
 2. `Relationship Name`
 3. `Master Account ID`
 4. `Master Account Name`
 5. `Account#`
 6. `Acct Name`
- **Row 1** will have these headers. Data begins on **Row 2**.

3. Call Logs

- **Not heavily used** in Phases 1 & 2, but it must exist to keep structure consistent.
- For now, simply create it with row 1 titled "Call Logs" or minimal headers like "Call Date," etc.
- We won't add code for call logging yet, but we do need the sheet present.

4. Reminders

- Similarly, this will be used more in Phase 3, but we still create it.
- Row 1 can have headers like "Next Call Due," "Reminder Interval," etc.

5. Change Log

- Tracks user actions, such as new account imports, overwrites, etc.
- Row 1 can have headers: `Timestamp | User | Action | Affected Data | Old Value | New Value`

Tip: The **exact** order of columns for "Call Logs" and "Reminders" does not matter for Phase 2. We will reference them in the code enumerations, but you only truly need to set up "Account Data" and "Change Log" columns precisely.

Creating These Sheets

1. **Add** (or rename) the first worksheet to "Dashboard."
2. **Add** four more worksheets with these exact names:
 - "Account Data"
 - "Call Logs"
 - "Reminders"
 - "Change Log"
3. On **"Account Data,"** enter the column headers on Row 1:

```
makefile
```

```
A1: Relationship#
B1: Relationship Name
C1: Master Account ID
D1: Master Account Name
E1: Account#
F1: Acct Name
```

(Leave row 2 and onward blank for imported data.)

4. On **"Change Log,"** enter the headers on Row 1:

vbnet

```
A1: Timestamp
B1: User
C1: Action
D1: Affected Data
E1: Old Value
F1: New Value
```

(Leave row 2 and onward blank.)

The other sheets can have placeholder column headers or remain mostly empty for now.

2. CREATING THE VBA MODULES

Open the Visual Basic Editor (press **ALT+F11** in Excel). You will create **4 modules** as follows. (Phase 1 and 2 only require 3 or 4 modules, but we'll keep them well-organized.)

1. **modGlobal**
2. **modUtilities**
3. **modDataUpload**
4. **modDashboard** (for the single "Upload Accounts" button handler)

We'll omit call/reminder modules for now, focusing only on phases 1 & 2.

Important: All code blocks below go into standard **modules** (not sheet modules and not userforms). In the VBE:

- **Insert** → **Module**
- Name it according to the section (e.g., "modGlobal")
- Paste in the code.

2.1 `modGlobal`

vb

Option Explicit

```
' -----
' modGlobal: Stores global constants, enumerations, and references to sheet names.
' -----
```

```
Public Const SH_DASHBOARD As String = "Dashboard"
Public Const SH_ACCOUNT_DATA As String = "Account Data"
Public Const SH_CALL_LOGS As String = "Call Logs"
Public Const SH_REMINDERS As String = "Reminders"
Public Const SH_CHANGE_LOG As String = "Change Log"
```

```
' We assume columns in "Account Data" appear in this order:
```

```
Public Enum AccountDataCols
    acRelNum = 1 ' Column A -> Relationship#
    acRelName ' Column B -> Relationship Name
    acMasterID ' Column C -> Master Account ID
    acMasterName ' Column D -> Master Account Name
    acAcctNum ' Column E -> Account#
    acAcctName ' Column F -> Acct Name
End Enum
```

```
' We assume columns in "Change Log" appear in this order:
Public Enum ChangeLogCols
    chgTimestamp = 1 ' Column A
    chgUser       ' Column B
    chgAction     ' Column C
    chgAffectedData ' Column D
    chgOldValue   ' Column E
    chgNewValue   ' Column F
End Enum
```

What this does:

- Defines the sheet names (so if you rename sheets later, you only fix it here).
- Defines enumerations so you can refer to columns by name instead of numbers (e.g., `acRelName`` instead of ``2``).

2.2 `modUtilities`

```
vb

Option Explicit

'-----
' modUtilities: Helper functions for row finding, logging changes, etc.
'-----

' Finds a row in "Account Data" where Relationship#, MasterAccountID, Account# match.
' Returns 0 if not found; otherwise returns row number.
Public Function FindAccountRow(ByVal relNum As String, _
                               ByVal masterID As String, _
                               ByVal acctNum As String) As Long

    Dim ws As Worksheet
    Dim lastRow As Long, i As Long

    Set ws = ThisWorkbook.Worksheets(SH_ACCOUNT_DATA)
    lastRow = ws.Cells(ws.Rows.Count, acRelNum).End(xlUp).Row

    For i = 2 To lastRow ' start from row 2 to skip headers
        If ws.Cells(i, acRelNum).Value = relNum _
            And ws.Cells(i, acMasterID).Value = masterID _
            And ws.Cells(i, acAcctNum).Value = acctNum Then
            FindAccountRow = i
            Exit Function
        End If
    Next i

    FindAccountRow = 0
End Function

' Highlights a given cell in the specified RGB color.
Public Sub HighlightCell(ByVal rng As Range, ByVal colorValue As Long)
    rng.Interior.Color = colorValue
End Sub

' Logs an action in "Change Log".
Public Sub LogChange(ByVal actionDesc As String, _
                     ByVal affectedData As String, _
                     Optional ByVal oldVal As String = "", _
                     Optional ByVal newVal As String = "")

    Dim ws As Worksheet
    Dim nextRow As Long

    Set ws = ThisWorkbook.Worksheets(SH_CHANGE_LOG)
    nextRow = ws.Cells(ws.Rows.Count, chgTimestamp).End(xlUp).Row + 1

    ws.Cells(nextRow, chgTimestamp).Value = Now
    ws.Cells(nextRow, chgUser).Value = Environ("Username") ' or Application.UserName
    ws.Cells(nextRow, chgAction).Value = actionDesc
```

```
ws.Cells(nextRow, chgAffectedData).Value = affectedData

If oldVal <> "" Then ws.Cells(nextRow, chgOldValue).Value = oldVal
If newVal <> "" Then ws.Cells(nextRow, chgNewValue).Value = newVal
End Sub
```

Explanation

- `FindAccountRow` checks if a specific `(Relationship#, Master Account ID, Account#)` combo exists.
- `HighlightCell` changes a cell's interior color to any `RGB(r, g, b)`.
- `LogChange` appends a new row to the "Change Log" sheet with timestamp, user, etc.

2.3 `modDataUpload`

```
vb

Option Explicit

'-----
' modDataUpload: Processes the uploaded workbook and merges new accounts into "Account Data."
'-----

Public Sub ProcessUpload(ByVal sourceFilePath As String, _
                        ByVal allowDuplicates As Boolean, _
                        ByVal overwrites As Collection)
    ' sourceFilePath: The file path the user selected
    ' allowDuplicates: True if the user wants to consider overwriting duplicates
    ' overwrites: A collection mapping "RelNum|MasterID|AcctNum" -> Boolean (True=overwrite)

    Dim wbSource As Workbook
    Dim wsSource As Worksheet
    Dim wsDest As Worksheet
    Dim lastRowSource As Long
    Dim i As Long

    Application.ScreenUpdating = False

    ' 1) Open the source workbook in read-only mode
    Set wbSource = Workbooks.Open(Filename:=sourceFilePath, ReadOnly:=True)
    Set wsSource = wbSource.Sheets(1) ' we assume data is on the first sheet
    Set wsDest = ThisWorkbook.Sheets(SH_ACCOUNT_DATA)

    ' 2) Determine the last used row in the source
    lastRowSource = wsSource.Cells(wsSource.Rows.Count, 1).End(xlUp).Row

    ' 3) Loop through each row in the source (assuming row 1 is headers)
    Dim relNum As String, relName As String
    Dim masterID As String, masterName As String
    Dim acctNum As String, acctName As String
    Dim existingRow As Long
    Dim destLastRow As Long

    For i = 2 To lastRowSource
        relNum = CStr(wsSource.Cells(i, 1).Value) ' Relationship#
        relName = CStr(wsSource.Cells(i, 2).Value) ' Relationship Name
        masterID = CStr(wsSource.Cells(i, 3).Value) ' Master Account ID
        masterName = CStr(wsSource.Cells(i, 4).Value) ' Master Account Name
        acctNum = CStr(wsSource.Cells(i, 5).Value) ' Account#
        acctName = CStr(wsSource.Cells(i, 6).Value) ' Acct Name

        existingRow = FindAccountRow(relNum, masterID, acctNum)

        If existingRow = 0 Then
            ' -> NEW ACCOUNT
            destLastRow = wsDest.Cells(wsDest.Rows.Count, acRelNum).End(xlUp).Row + 1

            wsDest.Cells(destLastRow, acRelNum).Value = relNum
            wsDest.Cells(destLastRow, acRelName).Value = relName
        End If
    Next i
End Sub
```

```

wsDest.Cells(destLastRow, acMasterID).Value = masterID
wsDest.Cells(destLastRow, acMasterName).Value = masterName
wsDest.Cells(destLastRow, acAcctNum).Value = acctNum
wsDest.Cells(destLastRow, acAcctName).Value = acctName

' Highlight new cells in a light-green color
HighlightCell wsDest.Cells(destLastRow, acRelNum), RGB(144, 238, 144)
HighlightCell wsDest.Cells(destLastRow, acRelName), RGB(144, 238, 144)
HighlightCell wsDest.Cells(destLastRow, acMasterID), RGB(144, 238, 144)
HighlightCell wsDest.Cells(destLastRow, acMasterName), RGB(144, 238, 144)
HighlightCell wsDest.Cells(destLastRow, acAcctNum), RGB(144, 238, 144)
HighlightCell wsDest.Cells(destLastRow, acAcctName), RGB(144, 238, 144)

' Log the new addition
LogChange "Import New Account", _
    "Rel#: " & relNum & "; Master: " & masterID & "; Acct: " & acctNum

' TODO: If you'd like to highlight the parent Relationship in yellow,
' you could add extra logic here to find existing rows with the same Relationship#,
' etc. For Phase 2, we focus on green highlight for new items.

Else
    ' -> DUPLICATE
    If allowDuplicates Then
        ' Check if user wants to overwrite
        Dim key As String
        key = relNum & "|" & masterID & "|" & acctNum

        If overwrites.Count > 0 Then
            If overwrites.Contains(key) Then
                If overwrites(key) = True Then
                    ' Overwrite existing data
                    Dim oldVal As String, newVal As String
                    oldVal = wsDest.Cells(existingRow, acRelName).Value & " / " & _
                        wsDest.Cells(existingRow, acMasterName).Value & " / " & _
                        wsDest.Cells(existingRow, acAcctName).Value

                    wsDest.Cells(existingRow, acRelName).Value = relName
                    wsDest.Cells(existingRow, acMasterName).Value = masterName
                    wsDest.Cells(existingRow, acAcctNum).Value = acctNum
                    wsDest.Cells(existingRow, acAcctName).Value = acctName

                    newVal = relName & " / " & masterName & " / " & acctName

                    LogChange "Overwrite Account", _
                        "Rel#: " & relNum & "; Master: " & masterID & "; Acct: " & acctNum,
                        oldVal, newVal

                    ' Highlight overwritten cells
                    HighlightCell wsDest.Cells(existingRow, acRelName), RGB(144, 238, 144)
                    HighlightCell wsDest.Cells(existingRow, acMasterName), RGB(144, 238, 144)
                    HighlightCell wsDest.Cells(existingRow, acAcctName), RGB(144, 238, 144)
                End If
            End If
        End If
    End If
End If
Next i

' 4) Close the source workbook without saving
wbSource.Close SaveChanges:=False

Application.ScreenUpdating = True
End Sub

```

Code Highlights

- ``ProcessUpload``:
 - Opens the user-selected file.
 - Reads each row (relationship, master, account).
 - Checks if it exists in "Account Data."

- If new, appends to the end and highlights in **green**.
- If a duplicate, can optionally overwrite if the user allows it.

2.4 `modDashboard`

We only need a **single** public procedure here for **Phase 2**: “Upload Accounts”. This macro is what you assign to the Dashboard’s button to **launch** the userform.

```
vb

Option Explicit

' -----
' modDashboard: Code for dashboard-related macros (Phase 1 & 2).
' -----

Public Sub Dashboard_UploadAccounts()
    ' Show the Upload UserForm
    frmUpload.Show
End Sub
```

That’s it. This keeps the code minimal for Phases 1 & 2.

3. CREATING AND CONFIGURING THE DASHBOARD

1. **Go to the “Dashboard” sheet** in Excel.
2. **Insert an ActiveX Command Button** (Developer Tab → Insert → ActiveX Controls → Command Button).
3. **Place** it anywhere on the dashboard.
4. **Right-click** the button and select **Properties**:
 - **Name** property: `cmdUploadAccounts` (optional but recommended for clarity).
 - **Caption** property: `Upload Accounts`.
5. **Assign the macro**: For an ActiveX control, you do this in the **button’s Click event**:
 - **Right-click** the button, choose **View Code**. You’ll see something like:

```
vb

Private Sub cmdUploadAccounts_Click()
    ' code goes here
End Sub
```

- **Inside** that click event, call our procedure:

```
vb

Private Sub cmdUploadAccounts_Click()
    Dashboard_UploadAccounts
End Sub
```

6. **Exit Design Mode** (on the Developer ribbon) to make the button clickable.

Now when you click “Upload Accounts,” it will call `Dashboard\_UploadAccounts` in `modDashboard`, which in turn will show the userform (`frmUpload`).

4. CREATING THE “UPLOAD ACCOUNTS” USERFORM

We need a userform named ``frmUpload`` that:

- Lets the user **browse** for a file.
- Shows a **preview** of new vs. duplicate accounts (optional but recommended).
- Lets the user choose if they **allow duplicates** (checkbox).
- Has an “**Import**” button that calls ``ProcessUpload``.

4.1 Step-by-Step: Creating ``frmUpload``

1. **Open VBE** (Alt+F11), then **Insert** → **UserForm**.
2. **Name** it ``frmUpload`` (in the Properties window under ``(Name)``).
3. **Add Controls** (in the Toolbox):
 1. **Label:**
 - ``(Name) = lblFilePath``
 - ``Caption = `` (leave blank)
 - Purpose: to display the selected file’s path.
 2. **CommandButton:**
 - ``(Name) = cmdBrowseFile``
 - ``Caption = "Browse..."``
 - Purpose: to open a file dialog and select the accounts file.
 3. **CheckBox:**
 - ``(Name) = chkAllowDuplicates``
 - ``Caption = "Allow Duplicates?"``
 4. **ListBox:**
 - ``(Name) = lstPreview``
 - ``ColumnCount = 6`` (in Properties)
 - Purpose: to preview the relationships, master IDs, accounts, and indicate “NEW” or “DUPLICATE.”
 5. **CommandButton:**
 - ``(Name) = cmdImport``
 - ``Caption = "Import"``
 - Purpose: to finalize the upload.

Resize and position the controls on the userform so it looks neat.

4.2 Add the Following Code to ``frmUpload``

Right-click the userform in the Project Explorer → **View Code**, then paste:

```
vb

Option Explicit

Private selectedFilePath As String
Private collOverwrites As Collection ' used if user chooses to overwrite duplicates

'-----
' The UserForm_Initialize or UserForm_Activate can reset form controls each time
'-----
Private Sub UserForm_Initialize()
    Me.lblFilePath.Caption = ""
    Me.chkAllowDuplicates.Value = False
    Me.lstPreview.Clear
```



```

Set collOverwrites = New Collection
End Sub

'-----
' Button: Browse File
'-----
Private Sub cmdBrowseFile_Click()
    Dim fd As FileDialog
    Set fd = Application.FileDialog(msoFileDialogFilePicker)

    With fd
        .Title = "Select the Accounts File"
        .Filters.Clear
        .Filters.Add "Excel Files", "*.xlsx; *.xlsm; *.csv", 1
        .AllowMultiSelect = False

        If .Show = -1 Then ' user clicked OK
            selectedFilePath = .SelectedItems(1)
            Me.lblFilePath.Caption = selectedFilePath

            ' Optionally load preview
            PreviewData selectedFilePath
        End If
    End With
End Sub

'-----
' Subroutine: PreviewData
' Reads the source file to identify "NEW" vs "DUPLICATE" and shows it in the ListBox.
'-----
Private Sub PreviewData(ByVal filePath As String)
    Dim wbTemp As Workbook, wsTemp As Worksheet
    Dim lastRow As Long
    Dim i As Long

    Application.ScreenUpdating = False
    Set wbTemp = Workbooks.Open(filePath, ReadOnly:=True)
    Set wsTemp = wbTemp.Sheets(1)

    lastRow = wsTemp.Cells(wsTemp.Rows.Count, 1).End(xlUp).Row

    Me.lstPreview.Clear

    ' We'll set column widths for better visibility (units: points)
    Me.lstPreview.ColumnCount = 6
    Me.lstPreview.ColumnWidths = "70;120;70;120;70;80"

    Dim relNum As String, relName As String
    Dim masterID As String, masterName As String
    Dim acctNum As String, acctName As String
    Dim existingRow As Long

    For i = 2 To lastRow
        relNum = CStr(wsTemp.Cells(i, 1).Value)
        relName = CStr(wsTemp.Cells(i, 2).Value)
        masterID = CStr(wsTemp.Cells(i, 3).Value)
        masterName = CStr(wsTemp.Cells(i, 4).Value)
        acctNum = CStr(wsTemp.Cells(i, 5).Value)
        acctName = CStr(wsTemp.Cells(i, 6).Value)

        existingRow = FindAccountRow(relNum, masterID, acctNum)

        Dim previewString As String
        If existingRow = 0 Then
            previewString = relNum & vbTab & relName & vbTab & masterID & _
                vbTab & masterName & vbTab & acctNum & vbTab & "*NEW*"
        Else
            previewString = relNum & vbTab & relName & vbTab & masterID & _
                vbTab & masterName & vbTab & acctNum & vbTab & "*DUPLICATE*"
        End If

        Me.lstPreview.AddItem previewString
    Next i

    wbTemp.Close False

```

```

Application.ScreenUpdating = True
End Sub

'-----
' Button: Import
'-----
Private Sub cmdImport_Click()
    If selectedFilePath = "" Then
        MsgBox "No file selected. Please browse for a file first.", vbExclamation
        Exit Sub
    End If

    ' Build an overwrites collection if you want more advanced logic to pick duplicates
    ' For now, we'll pass an empty collection or a dummy one.
    Set collOverwrites = New Collection

    Dim allowDup As Boolean
    allowDup = Me.chkAllowDuplicates.Value

    ProcessUpload selectedFilePath, allowDup, collOverwrites

    MsgBox "Accounts have been uploaded successfully!", vbInformation

    Unload Me
End Sub

```

Explanation

- ``cmdBrowseFile_Click`` opens a file dialog so the user can pick an external file.
- ``PreviewData`` reads the first sheet of that file and quickly checks each row to see if it's new or duplicate. Displays the result in the ``lstPreview``.
- ``cmdImport_Click`` calls ``ProcessUpload`` from ``modDataUpload``, passing the file path and the user's duplicates preference.

FULL WORKFLOW TEST

1. **Open your main workbook** (with the 5 sheets set up).
2. **Press ALT+F11**, ensure you have the 4 modules:
 - ``modGlobal``, ``modUtilities``, ``modDataUpload``, ``modDashboard``.
 - ``frmUpload`` userform with the correct controls/code.
3. **Go to the "Dashboard" sheet.** Ensure you have an ActiveX button named ``cmdUploadAccounts``, and its **Click** event calls ``Dashboard_UploadAccounts``.
4. **Exit design mode.**
5. **Click the button "Upload Accounts."**
 - The userform ``frmUpload`` appears.
 - Click "Browse..." to select an external Excel or CSV file.
 - The userform shows a preview listing which lines are *"NEW"* vs. *"DUPLICATE."*
6. **Check "Allow Duplicates?"** if you wish to allow overwriting. (For this basic example, overwriting is only automatic if you coded more logic in the userform, but the placeholder is there.)
7. **Click "Import."**
 - The system closes the external file.
 - Any new accounts go to **"Account Data"** and are highlighted green.
 - A log entry is made in **"Change Log."**
8. **Inspect** your "Account Data" sheet to confirm new rows were added.
9. **Inspect** your "Change Log" sheet to confirm the logged actions.

Final Notes & Next Steps

- **Performance Enhancements:** For very large files, you might load data into arrays instead of row-by-row.
- **Parent Highlighting:** Currently, we only highlight **new** accounts in **green**. If you want a **parent** Relationship or Master row to be in **yellow** (when a child is newly added), add logic to search existing Relationship or Master in ``ProcessUpload``.
- **Duplicate Overwrites:** The code supports overwriting if you pass in a ``collOverwrites`` that marks each duplicate as ``True``. The sample userform does not fully let the user pick duplicates to overwrite individually—but the structure is there. You can expand on that if needed.
- **Change Log:** Overwrites also log the ``oldVal`` vs. ``newVal``. If you do partial overwrites or more advanced merges, adapt it accordingly.

By carefully following these instructions for **Phase 1** (Workbook Setup & Core Modules) and **Phase 2** (Upload & Preview Accounts), you will have a fully functional, **modular** foundation to manage and import account data in Excel VBA.